



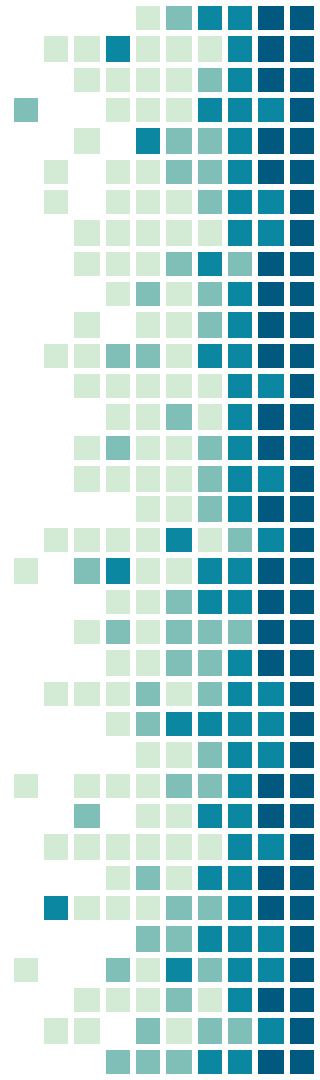
CdL in Informatica – A.A. 2024 – 2025

Programmazione 1 – Modulo 2

Lezione 8
29/10/2024

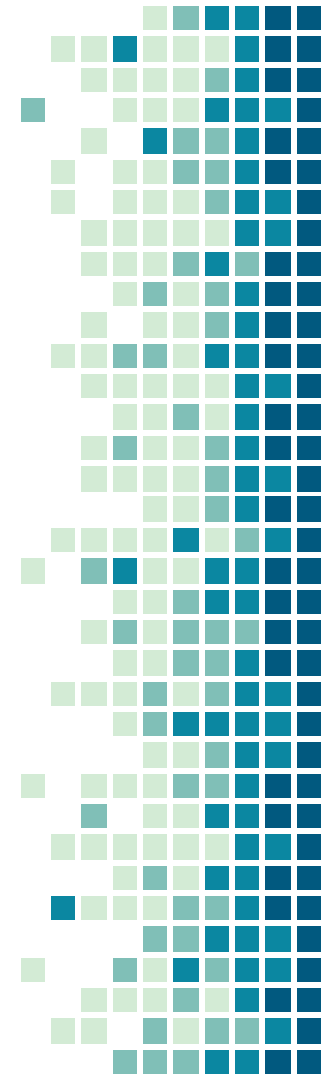
Andrea Loddo

Federico Meloni - Alessandra Perniciano - Fabio Pili



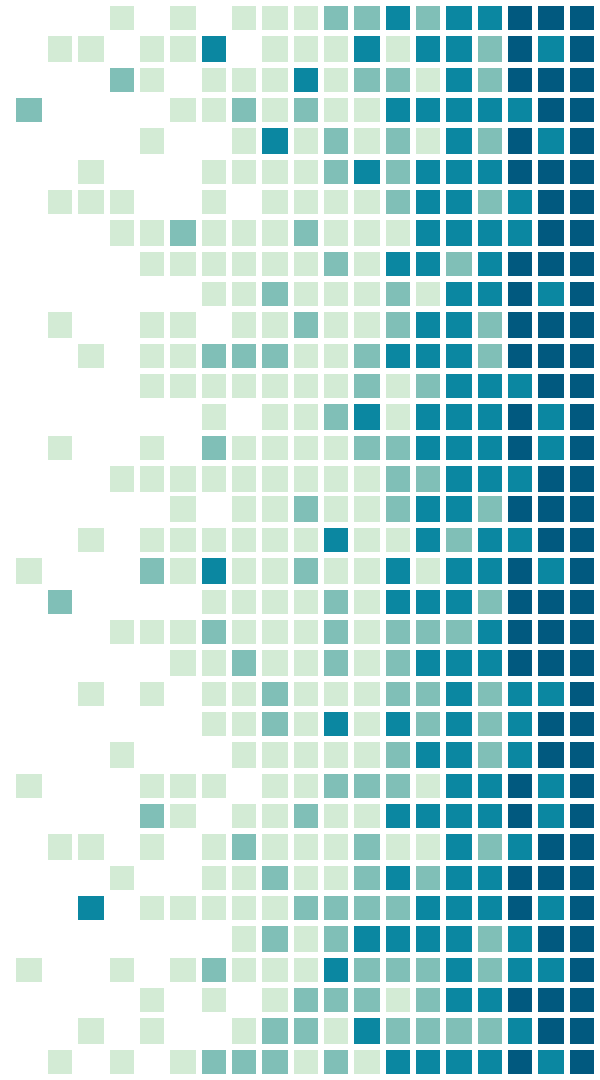
Argomenti

- Array di caratteri (stringhe)
 - Memorizzazione
 - Inizializzazione
 - Inserimento
 - Stampa stringhe
- Libreria string.h
 - Copia
 - Lunghezza
 - Concatenazione
 - Comparazione



Tipi di dato non primitivi

Gli array di caratteri



Stringhe (array di caratteri)

- Come per i tipi numerici, è possibile aggregare variabili di tipo char in una singola struttura dati: array di char (stringa).
- Una stringa si dichiara come un array qualsiasi
 - con la ovvia particolarità del tipo char
 - con una dimensione costante

```
/* Dichiarazione stringa di dimensione DIM */  
char nome[DIM];
```

- Linguaggio C: non dispone di un tipo di dato nativo per gli insiemi di caratteri
 - un buon numero di operazioni è implementato nella libreria string
- Le stringhe in C hanno molteplici peculiarità, legate a:
 - memorizzazione
 - inizializzazione
 - input e output

Stringhe: memorizzazione

- Negli array numerici, i valori sono memorizzati normalmente:

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

- Nelle stringhe:
 - gli elementi sono memorizzati normalmente ma
 - dopo l'ultimo elemento, viene aggiunto automaticamente il carattere di escape '\0' (carattere null)

P	r	o	g	r	a	m	m	a	\0
---	---	---	---	---	---	---	---	---	----

- Se si vogliono memorizzare N caratteri, è buona norma specificare una stringa di dimensione N+1 per consentire la memorizzazione del carattere null:

```
#define DIM_STRINGA 30
...

char squadra[DIM_STRINGA+1];
```

Stringhe: inizializzazione

- Array: necessario specificare i singoli elementi uno per volta;
- Stringhe: possono essere inizializzate con una singola istruzione e i caratteri che ne fanno parte sono racchiusi tra doppi apici.



```
char squadra[] = "Cagliari Calcio";
```

La dimensione (15 + 1) viene calcolata automaticamente dal compilatore (spazi inclusi: sono caratteri come gli altri).



```
char squadra[25] = "Dinamo Sassari";
```

Si usano soltanto 14 caratteri. I restanti 11 vengono riempiti con '\0'.



```
char squadra[8] = "Cagliari";
```

Si usano tutti gli 8 caratteri ma non c'è spazio per '\0'. Cosa succede se proviamo a visualizzare la stringa?



```
char squadra[4] = "Cagliari";
```

Si dichiara una stringa di 4 caratteri ma in fase di inizializzazione si specifica una stringa più lunga. Cosa succede se proviamo a visualizzare la stringa?

- Quando non viene memorizzato il carattere null a causa di una inizializzazione con dimensione insufficiente, la stringa non è utilizzabile in modo sicuro (undefined behaviour).

Stringhe: inserimento

- È possibile acquisire tutti gli elementi di una stringa con una sola istruzione di input:

- NB: stringa non necessita di &

```
char stringa[20];  
scanf("%s", stringa);
```

- L'istruzione:
 - legge una sequenza di caratteri finché l'utente non preme il tasto "Invio", o fino a un carattere di spaziatura
 - non legge eventuali spazi: viene memorizzata solo la parola prima dello spazio.
- Per memorizzare anche gli spazi:

```
scanf("%[^\n]s", stringa);
```

```
gets(stringa);
```

Stringhe: inserimento

- Cosa succede se l'utente inserisce più caratteri di quelli disponibili?

```
char stringa[10];  
/* Inserisco "Dipartimento" */  
scanf("%s", stringa);  
/* stringa conterrà "Dipartimen" senza '\0' */
```

- È possibile limitare il numero di caratteri letti dalla scanf utilizzando "%ns";
- n è un numero intero che indica il numero massimo di caratteri letti dalla scanf

```
char stringa[10];  
scanf("%9s", stringa);
```

- È anche possibile utilizzare una MACRO per indicare il numero massimo di caratteri:

```
#define DIMSTRING 10  
#define DIMSINPUT "10"  
char stringa[DIMSTRING+1];  
scanf("%" DIMSINPUT "s", stringa);
```

- In generale, l'inserimento di stringhe può comportare dei problemi legati al fatto che il C non effettua alcun controllo sulla dimensione della stringa, per cui si potrebbe "sforare" e scrivere in zone di memoria contigue a quelle riservate alla stringa.

Stringhe: stampa

- È possibile stampare un'intera stringa con un solo comando, a differenza degli array numerici:

```
char stringa[10] = "ciao";  
printf("%s", stringa);
```

```
char stringa[10] = "ciao";  
puts(stringa);
```

- Naturalmente, essendo un array di caratteri, è sempre valida una stampa elemento per elemento
- Il while deve verificare se si è raggiunto il carattere di fine stringa oppure se si è oltrepassata la dimensione della stringa.

```
char stringa[10] = "ciao";  
int i = 0;  
  
while( stringa[i] != '\0' && i<10 )  
{  
    printf("%c", stringa[i]);  
    i++;  
}
```

La libreria string

- Il C non permette di usare direttamente operatori sulle stringhe (al contrario dei tipi predefiniti – int, char, float...)
- Le stringhe sono array a tutti gli effetti, e in quanto tali soggette agli stessi limiti.
- Un modo molto più comodo per effettuare operazioni sulle stringhe è avvalersi della libreria string.h e delle relative funzioni.
- Le più comunemente utilizzate sono:
 - strcpy(stringa1, stringa2) → copia stringa2 in stringa1
 - strlen(stringa) → restituisce la lunghezza della stringa
 - strcat(stringa1, stringa2) → concatena le due stringhe
 - strcmp(stringa1, stringa2) → comparazione alfabetica

```
/* non sono consentite: */  
stringa1 == stringa2;  
stringa1 += stringa2;
```

```
#include <string.h>
```

Stringhe: copia

```
#include <string.h>
```

```
/* Prototipo della funzione */  
strcpy(char[] s1, char[] s2)
```

- strcpy riceve due stringhe e copia il contenuto di s2 in s1, sovrascrivendolo.
- Non effettua controlli sulla lunghezza: verificare prima che s1 sia in grado di contenere s2.

- Esempio:

```
char s1[15] = "Analisi";  
char s2[15] = "Matematica";  
strcpy( s1, s2 );
```

- Cosa conterranno s1 e s2?
- Possiamo limitare il numero di caratteri da copiare usando un'altra funzione aggiungendo un parametro:

```
strncpy( s1, s2, max );
```

- Copierà soltanto i primi max caratteri di s2 sulla stringa s1.

Stringhe: lunghezza

```
#include <string.h>
```

```
/* Prototipo della funzione */  
int strlen(char[] s)
```

- La funzione strlen riceve una stringa e restituisce un valore intero, rappresentante la lunghezza della stringa (il carattere '\0' non viene conteggiato).

- Esempio:

```
char s[20] = "Informatica";  
int len = strlen( s );  
printf("%d\n", len);
```

- Quanto vale len?

Stringhe: concatenazione

```
#include <string.h>
```

```
/* Prototipo della funzione */  
strcat(char[] s1, char[] s2)
```

- La funzione strcat riceve due stringhe e aggiunge il contenuto della stringa s2 alla stringa s1.
- Non effettua controlli sulla lunghezza: verificare prima che s1 sia in grado di contenere la stringa risultante.

- Esempio:

```
char s1[20] = "Cagliari";  
char s2[20] = " Calcio";  
strcat( s1, s2 );
```

- Cosa conterranno s1 e s2?
- Anche qui, possiamo limitare il numero di caratteri da aggiungere in coda a s1 usando un'altra funzione e un terzo parametro:

```
strncat( s1, s2, num );
```

- Aggiungerà soltanto i primi num caratteri di s2 sulla stringa s1.

Stringhe: comparazione

```
#include <string.h>
```

```
/* Prototipo della funzione */  
int strcmp(char[] s1, char[] s2)
```

- La funzione strcmp riceve due stringhe, le confronta e restituisce un valore intero con i seguenti significati:
 - 0 se le stringhe sono uguali;
 - un valore minore di 0 se s1 è minore di s2 (alfabeticamente precedente);
 - un valore maggiore di 0 se s1 è maggiore di s2 (alfabeticamente successiva).
- Effettua un ordinamento lessicografico (ordinamento del dizionario).
- Esempio:

```
char s1[20] = "Cagliari";  
char s2[20] = "Sassari";  
int i = strcmp(s1, s2);  
int j = strcmp(s2, s1);  
int k = strcmp(s1, s1);
```
- Quanto varranno i, j, k ?

Array di stringhe

- Una stringa è un array di caratteri
- Un array di stringhe è una **matrice di caratteri**, in cui **ogni riga corrisponde ad una stringa**.

```
/* Dichiarazione array stringhe */  
char arrayStringhe[R][C];
```

- È possibile **inizializzare** un array di stringhe in fase di dichiarazione, in maniera simile alle matrici.
NB: ogni riga corrisponde ad una stringa!

```
/* Inizializzazione array stringhe */  
char arrayStringhe[R][C] = {"Cagliari", "Sassari", "Oristano"};
```

Array di stringhe: input e output

- Valgono tutte le considerazioni relative agli input/output visti per le stringhe, con l'unica eccezione che si accede una stringa mediante indice.

- Esempio di input:

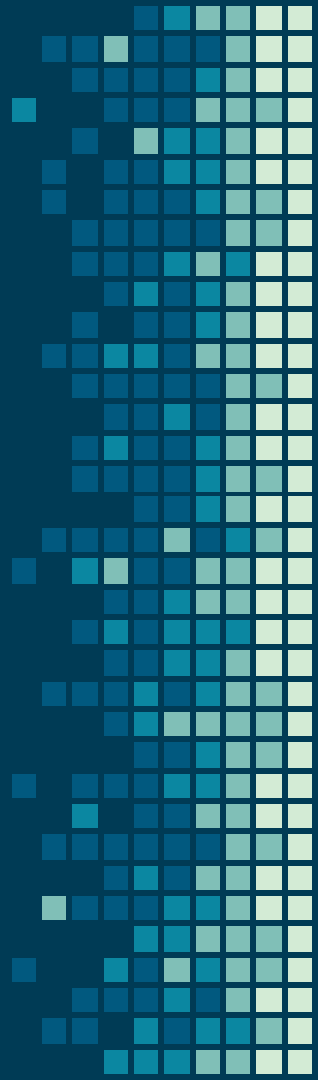
```
char arrayStringhe[R][C];  
for( i=0; i<R; i++ ){  
    scanf("%s", stringa[i]);  
}
```

- Esempio di output:

```
char arrayStringhe[R][C];  
for( i=0; i<R; i++ ){  
    printf("%s", stringa[i]);  
}
```

FINE!

Domande?



Autovalutazione

1. Che differenza c'è tra un array di caratteri e una stringa?
2. Con quale tipo deve essere dichiarata una stringa?
3. Il tipo stringa è un tipo nativo del C?
4. Considerato che un char vale 1 byte, quanto occupa in memoria la stringa definita `char nome[11];` ?
5. Cosa rappresenta e a cosa serve il carattere di escape null (`'\0'`)?
6. Come si inizializza una stringa?
7. È possibile definire e inizializzare una stringa senza specificare la sua dimensione?
8. Quale segnaposto si usa per stampare e acquisire una stringa?
9. È possibile stampare il contenuto di una stringa utilizzando un costrutto iterativo? Se sì, come?
10. È possibile limitare il numero di caratteri da assegnare ad una stringa mediante acquisizione? Come?
11. A cosa serve la libreria `string`?
12. Come si può effettuare la copia di una stringa?
13. Come si può calcolare la lunghezza di una stringa?
14. Come si possono concatenare due stringhe?
15. Come si possono comparare alfabeticamente due stringhe?

Esercizi

1. Scrivere un programma che date due stringhe in input stampi la più lunga. Se sono di uguale lunghezza, stamperà solo la prima acquisita.
2. Scrivere un programma che chieda in input una stringa e calcoli da quanti caratteri è composta, senza usare `strlen`.
3. Scrivere un programma in cui si dichiarano due stringhe di dimensione `N`, definita mediante direttiva `#define`, e si acquisiscono entrambe in input. Il programma deve copiare la seconda stringa all'interno della prima, senza usare la `strcpy`. Stampare le stringhe prima e dopo l'operazione di copia.
4. Scrivere un programma che date due stringhe in input stampi la maggiore alfabeticamente, sia con funzioni di libreria che senza.
5. Scrivere un programma che acquisisca una stringa e la modifichi trasformando tutti i caratteri minuscoli in maiuscoli e viceversa. Tutti gli altri caratteri vanno lasciati inalterati.
6. Scrivere un programma che acquisisca una stringa e conti il numero di caratteri spazio inseriti dall'utente, in due versioni diverse:
 1. utilizzando la `strlen` (utilizzando un `FOR`),
 2. con un ciclo che si ferma quando incontra il carattere di escape `'\0'` (utilizzando un `WHILE`).Quale soluzione sarebbe preferibile? Perché?
7. Scrivere un programma che cerchi la "maggiore" e la "minore" tra una serie di parole inserite dall'utente.
8. Scrivere un filtro `B1FF`, che legga una stringa inserita dall'utente e la modifichi secondo le seguenti regole:
 - tutte le lettere diventano maiuscole;
 - le seguenti lettere vengono trasformate nei seguenti numeri: A->4, B->8, E->3, I->1, O->0, S->5;
 - il messaggio finisce sempre con la sequenza: `"..!!1!1"`.

Esercizi

9. Scrivere un programma che prenda in ingresso due stringhe, s1 ed s2, e stampi se la stringa s2 è completamente contenuta in s1 o meno. Per "completamente contenuta" si intende che i caratteri di s2 devono essere presenti in s1 nella stessa sequenza in cui compaiono in s2. Ad esempio, la stringa "zio" è contenuta in "funzione".
10. Scrivere un programma che prenda in ingresso una stringa e stampi in output se la stringa è palindroma. Una stringa è palindroma se può essere letta nello stesso modo da sinistra verso destra e da destra verso sinistra. Esempi: "anna", "kayak", "elle", "otto" sono palindrome.
11. Scrivere un programma che prenda in ingresso una stringa e conti quante e quali lettere si ripetono. Stampare in output il conteggio.
12. Scrivere un programma che richieda in input una stringa e conti di quante lettere maiuscole, lettere minuscole, cifre e altri caratteri è composta.
Esempio "Buongiorno2023! 6Qui?" deve fornire in output: maiuscole:2, minuscole: 11, cifre: 5, altri: 3.
13. Scrivere un programma che chieda all'utente di inserire 3 frasi, e le salvi in un array di stringhe. Chiedere poi all'utente quale delle 3 frasi stampare, quindi stampare la frase desiderata.
14. Scrivere un programma in cui si acquisiscono in input S stringhe di dimensione N. S e N sono definite mediante direttiva #define. Le stringhe acquisite vanno salvate in un array di stringhe. Il programma deve cercare e stampare la "maggiore" e la "minore" tra le stringhe inserite dall'utente. Hint: modificare l'esercizio 4.
15. Scrivere un programma che chieda all'utente di inserire una frase (max 128 caratteri), conti quante sono le parole (sequenze di lettere dell'alfabeto) che la compongono e la lunghezza media delle parole stesse.
Esempio: se viene inserito in input: "Ieri... sono andato a mangiare all'una!", il programma deve indicare che ci sono 7 parole e che la lunghezza media è 4.14 caratteri.

